Replaying the Shift

How to find out what really broke a robotic sort floor, and what it cost

KASSEM KANJO · OCTOBER 2026 · SWITCHYARD v1.0

IN SHORT When a sort hub suddenly loses a third of its throughput, nobody can say for sure what caused it or what each problem cost. I built a digital twin of a robot sort floor that answers that by *replaying the shift*: it re-runs the same day with each problem removed and measures the difference. It names the right cause in 24 of 24 planted incidents and 44 of 48 hard ones, says "we can't tell" when it can't, and it exposed a common mistake that makes bad fixes look good.

The problem

A modern hub runs a few hundred robots carrying parcels from induction stations to chutes. When something breaks, a robot dies in an aisle or a chute jams, the damage doesn't stay where it happened. It spreads as traffic through robots that never failed. By the time the throughput chart drops, the alarm points at the symptom. When two or three things go wrong in the same ten minutes, which is normal on a busy shift, nobody can say how much each one cost. So the fix that gets funded is whoever argues best in the meeting.

The tools on the market don't answer this question. Layout simulators tell you if a design will work before go-live. Fleet dashboards tell you what one robot is doing. None of them tell you *why we lost 31 % at 14:02, and what each cause was worth*.

The idea

Software companies solved this years ago. When a large web service goes down, engineers trace every request, break things on purpose to find weak spots, and write a post-mortem that says exactly what failed and what to change. My starting point was simple: **a sort floor is a distributed system**. Robots are the servers, aisles are shared resources, a jam is an outage, and a parcel is a request with a deadline. If that's true, the whole software toolkit should work on a warehouse. Switchyard is that bet.

How it works

A twin you can rewind. Switchyard simulates the floor robot by robot, and it is fully deterministic: the same inputs always give exactly the same shift, on any computer. That sounds like a technical detail, but everything depends on it. If the day always plays out the same way, then any difference between two runs comes from the one thing I changed. Every robot that waits also records *why* it waited, so every late parcel gets a blame chain like "41 s stuck: behind R112, which was behind R087, which was stuck behind dead robot R031."

Replay the day without each problem. When throughput drops, Switchyard finds the incident, lists every fault that was active, and replays the shift with each combination of those faults removed. Each replay shows how much delay that removal would have saved. Delay is measured in parcel-minutes: one parcel one minute late is one parcel-minute. The blame is then split fairly with the Shapley value from game theory, the standard way to divide credit between players who worked together. The same replays show whether two faults overlapped (fixing one makes the other matter less) or compounded (together they did more damage than apart). Candidate fixes, like rerouting a chute or clearing dead robots faster, are tested the same way, on the real day.

Honest error bars from a system with no randomness. This is the part I'm proudest of. A deterministic twin gives one exact answer, which looks precise but isn't. Remove a fault and the robots take slightly different paths, so a saving of 947 parcel-minutes could just as easily have been 900 or 1,000. To measure that, I make the twin generate its own alternative days: I replay the real shift several times with one robot

pausing for half a second at a moment that shouldn't matter, and keep only the copies that actually play out differently. Every answer is then measured across all of those days. The spread is the error bar. If two causes are within it, the report calls them a tie, and if a fix is within it, the report says the fix did nothing measurable.

What it found

Figure 1 is one real incident: a chute jams and two robots die in the same minute. Throughput drops 47 %, the incident costs 6,264 parcel-minutes, and 110 parcels end up more than ten minutes late. Switchyard runs 96 replays and writes the report in under four seconds.

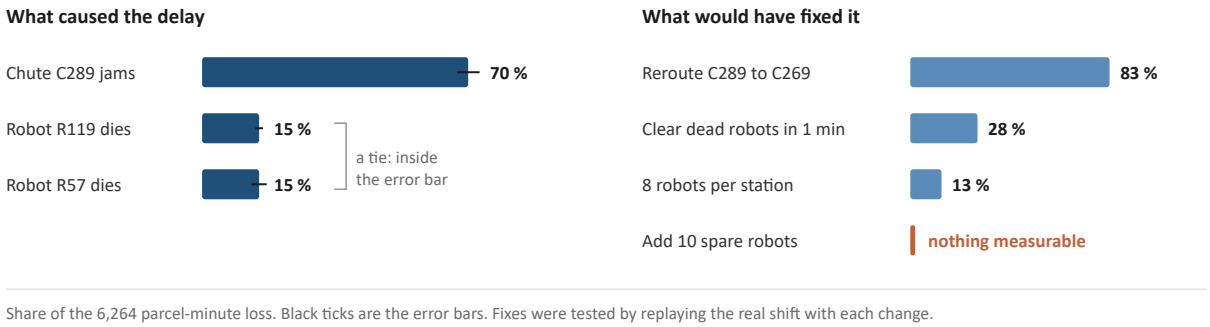


Figure 1. One incident, explained. The jam caused most of the delay. The two robot failures are too close to rank, so the report doesn't pretend to. The usual reflex, sending in more robots, would have done nothing here, because the bottleneck was a chute, not the fleet. A free procedure change, clearing dead robots in one minute instead of five, was worth 28 %.

Does it get the answer right? On 24 incidents where I planted one clear cause, it named that cause every time. That test is too easy, so I built a hard one: 48 incidents made of fault pairs deliberately chosen to be close, overlapping or compounding. Each one is graded against a separate check that simulates every fault alone and together and never looks at the report. Grading reads the actual text a site manager would see.



Figure 2. Right when it matters, honest when it can't know. Big effects are read correctly every time. Small interactions, the ones worth under about 500 parcel-minutes, sit inside the floor's own noise, so no method that replays this floor could settle them. My first version of this test scored 100 %, because I had quietly swapped out the two kinds of case the engine got wrong. That is exactly the kind of mistake the engine exists to catch, so I put them back.

Averages invent fixes. The most useful finding came from a bug. An early report said rerouting the jammed chute saved *more* than removing the jam itself: 5,162 against 5,076 parcel-minutes. That's the kind of surprising result that gets presented in a meeting and turned into policy. It wasn't real. The two numbers were averages taken over different versions of the day. Compared day by day, the two options tie (Figure 3).

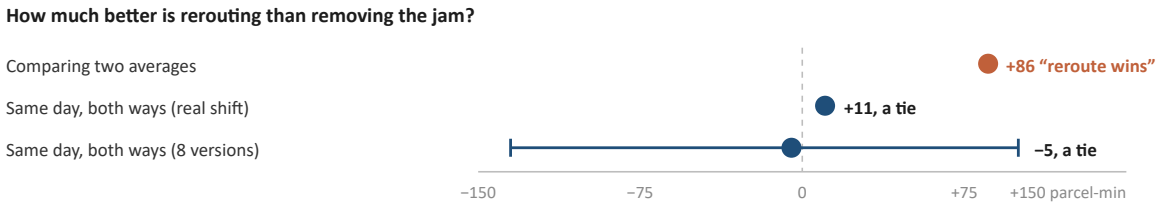


Figure 3. The same question, answered two ways. Comparing two separate averages invents an 86 parcel-minute win. Running the same day both ways shows a tie: each option wins four of the eight versions, and the line shows the full range.

This goes well beyond warehouses. Any team that compares “days we did A” with “days we did B” can manufacture a fix that doesn’t exist. The cure is what Switchyard does: judge a change on the same day, played both ways.

What this means for a hub

- **Where a fault happens matters as much as what it is.** Two identical outages at different induction stations cost 3,930 and 4,690 parcel-minutes. A single maintenance target for every station is the wrong contract.
- **Gut feel misjudges combined incidents.** I expected an outage and a surge at the same station to overlap. They compounded, eight times out of eight.
- **A calm floor can be closer to the edge than it looks.** Most small hiccups change nothing because the robots route around them. That resilience is real, but it also hides how little margin is left.
- **Some questions can’t be answered, and a good report says so.** If two small problems are inside the noise, any ranking of them is a guess.

Building it with AI

Switchyard needed five fields at once: robot path-planning, the way software teams run incidents, game theory for splitting blame, cause-and-effect statistics, and the alarm methods factories use to spot when a process goes off track. A normal team would need an expert in each, and the core idea only shows up when all five are in one head at the same time. AI gave me that. I owned the idea and the judgement; AI gave me working depth in every field whenever I needed it.

I also ran the build like a small company of AI agents. Cheap models wrote well-defined code, stronger models reviewed it, and an independent auditor reported only to me. The auditor caught real mistakes, including labels that were the wrong way round and a headline number that quietly showed an average instead of what actually happened. But the AI was also the one that rigged the first test to score 100 %, and it took a human asking “does this test actually test anything?” to catch it. AI widens what one person can build across fields. It doesn’t replace someone deciding what counts as true.

What’s next

Today the floor is simulated. The next step is to connect the twin to a real hub’s logs, so the same replays run on a copy of an actual shift. After that, the focus is the report itself: one plain sentence at the top that a shift manager can act on in ten seconds.

Code, design records and every test in this paper: github.com/kassem-kanjo/switchyard (v1.0). Robot planning: Okumura et al., PIBT, *Artificial Intelligence*, 2022. Blame splitting: Shapley, “A value for n-person games”, 1953; interactions: Grabisch & Roubens, 1999. Incident alarms: Page, CUSUM, *Biometrika*, 1954. Post-mortem practice: Beyer et al., *Site Reliability Engineering*, O’Reilly, 2016.